PRÄSENTATION FÜR EIN PYTHON-TUTORIUM

Überblick von Maschinensprache bis dynamische Typisierung

MASCHINENSPRACHE

Ebene
Niedrigste Software-Ebene – direkt unterhalb jeder anderen Programmiersprache.

Eigenschaften
Besteht rein aus Binärcode (0en und 1en). Jeder Befehl ist eine Folge von Bits, die der Prozessor direkt interpretiert.

Unterschiede zu höheren Sprachen
Direkt von der CPU ausführbar – kein Übersetzer notwendig. Für Menschen jedoch extrem schwer zu lesen, zu schreiben oder zu debuggen. Stark abhängig von der jeweiligen Hardware-Architektur: Ein Programm in Maschinensprache läuft nur auf der CPU-Familie, für die es kompiliert wurde.

ASSEMBLERSPRACHE

Ebene:
Maschinennahe Sprache – eine Abstraktionsstufe über dem reinen Binärcode.

Eigenschaften:
Verwendet kurze Text-Kürzel (Mnemonic-Codes) anstelle von Binärzahlen.
Beispiele: MOV, ADD, JMP

Unterschied zur Maschinensprache:
Besser für Menschen lesbar und verständlich als reiner Binärcode.
Dennoch besteht eine 1:1-Beziehung zu den Befehlen des Prozessors – jeder Assembler-Befehl entspricht genau einem Maschinenbefehl.

Übersetzung:
Ein Assembler übersetzt die Mnemonic-Codes direkt in Maschinencode, der von der CPU ausgeführt werden kann. Die Sprache bleibt stark hardwareabhängig.

COMPILERSPRACHEN VS. INTERPRETER-SPRACHEN

Übersetzungsart

Compilersprachen:
Vollständige Übersetzung des gesamten Quellcodes vor der Ausführung.
Interpreter-Sprachen:
Zeilenweise Übersetzung und Ausführung zur Laufzeit.

Ausführungsgeschwindigkeit

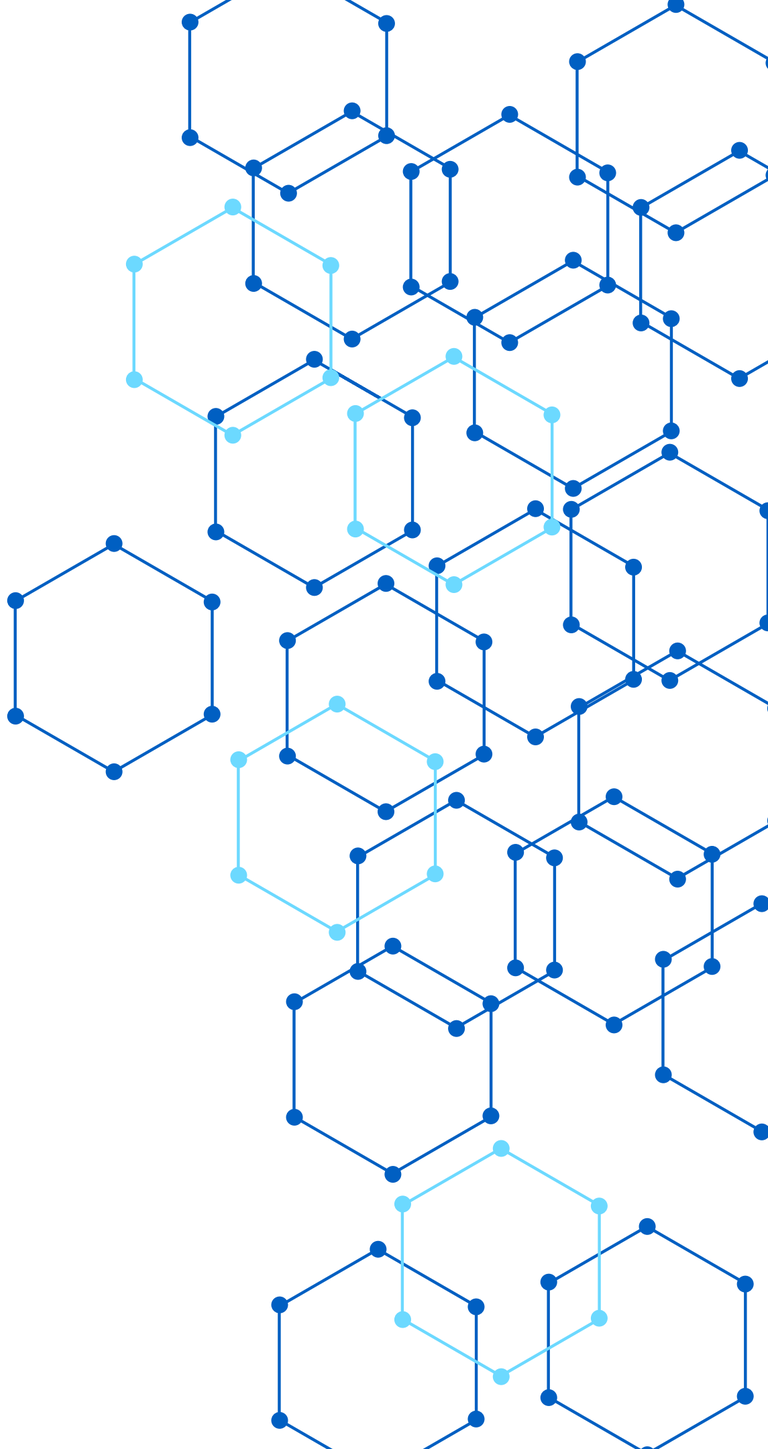
Compilersprachen: Schnelle Verarbeitung, da der Code bereits vollständig übersetzt wurde.
Interpreter-Sprachen:
Langsamere Verarbeitung durch die schrittweise Übersetzung zur Laufzeit.

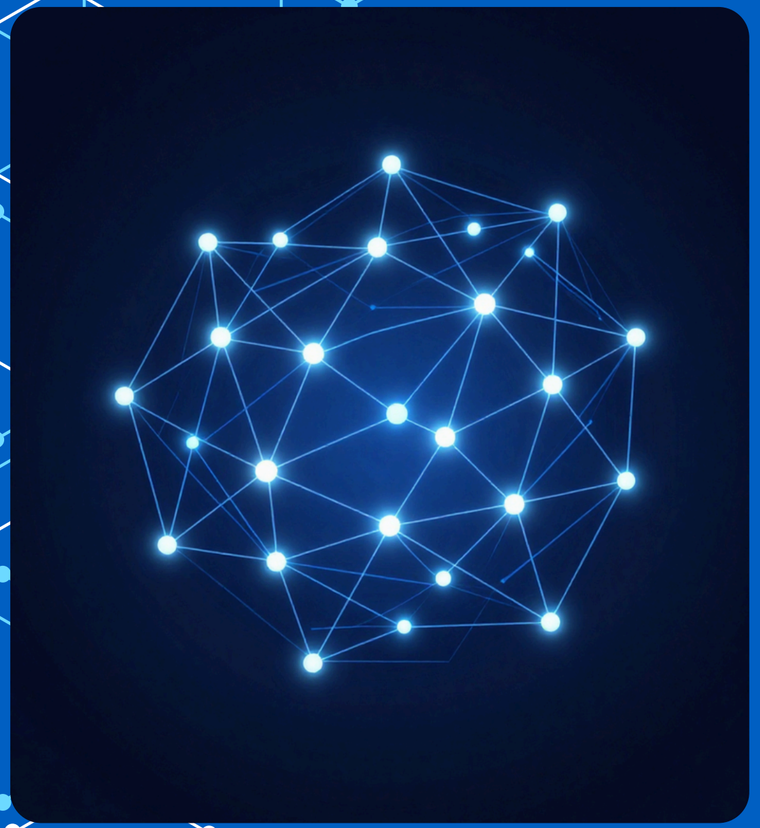
Plattformunabhängigkeit

Compilersprachen:
Plattformabhängiger Maschinencode – muss für jede Zielarchitektur neu kompiliert werden.
Interpreter-Sprachen:
Plattformunabhängig durch den Einsatz eines Interpreters.

VON ALL DIESEN SPRACHEN LERNEN WIR NUR PYTHON

Python ist also sowohl eine Compilersprache als auch eine Interpretersprache.
Python-Code wird zunächst durch einen internen Compiler in einen plattformunabhängigen Bytecode übersetzt. Dieser Bytecode wird anschließend von der Python Virtual Machine (PVM) zeilenweise interpretiert und ausgeführt.
Dieses hybride Modell vereint die Vorteile beider Welten: Portabilität durch den Interpreter und strukturierte Übersetzung durch den Compiler.





ALLES IST EIN OBJEKT

Jedes Element in Python ist ein eigenständiges Objekt.
Daten + Funktionen = Objekt
(Attribute + Methoden)
Kapselung schützt interne Details, hält den Code sauber und macht ihn weniger fehleranfällig.

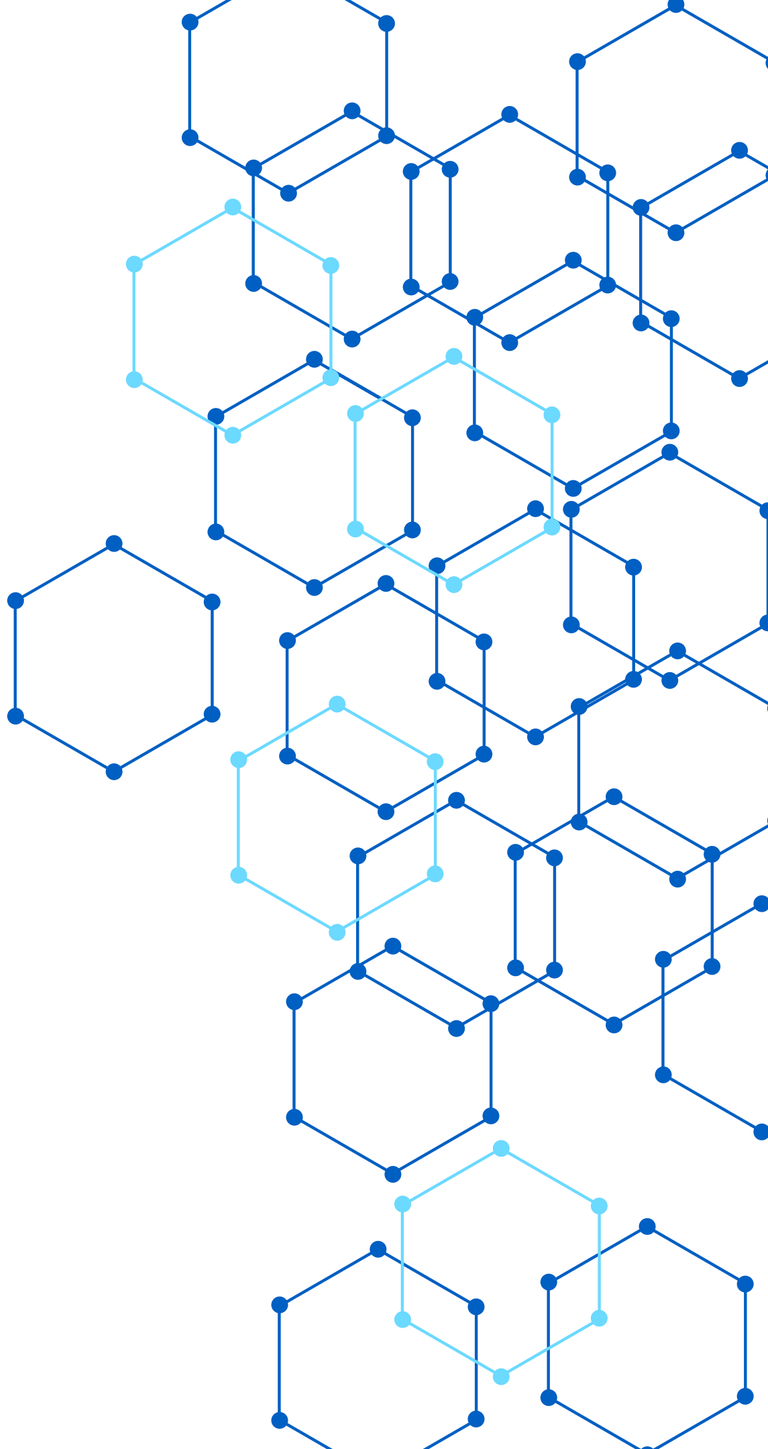


BITFOLGEN

Computer stellen Informationen als Bitfolgen dar.
Bit: Kleinste Informationseinheit – kann nur zwei Zustände annehmen: True / False (1 oder 0).
Byte: 8 Bits zusammengefasst ergeben ein Byte – die grundlegende Speichereinheit moderner Computer.

KONVERTIERUNG: DEZIMAL ↔ BINÄR

Dezimal → Binär (Division durch 2):
13 ÷ 2 = 6 Rest 1
6 ÷ 2 = 3 Rest 0
3 ÷ 2 = 1 Rest 1
1 ÷ 2 = 0 Rest 1
→ Ergebnis (von unten lesen): 1101_2
Binär → Dezimal (Stellenwerte):
$1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$
$= 8 + 4 + 0 + 1 = 13_{10}$
In Python:
bin(13) → '0b1101'
int('1101', 2) → 13



TYPEN VON ZAHLEN IN PYTHON

- bool: Wahrheitswerte True / False, intern als 1 und 0 gespeichert. Subtyp von int. Ergebnis von Vergleichs- und logischen Operationen.
- int: Ganze Zahlen beliebiger Länge ohne feste Größenbeschränkung. Unterstützt alle arithmetischen Operationen. Kein Überlauf in Python.
- float: 64-Bit Fließkommazahlen nach IEEE 754. Sonderfälle: Underflow → kleinste darstellbare Zahl wird zu 0.0; Overflow → Ergebnis wird zu inf.

SPEICHERUNG IM 64-BIT-SPEICHER



- Mantisse: 52 Bit – speichert die signifikanten Ziffern der Zahl
- Exponent: 11 Bit – bestimmt die Größenordnung (Skalierung)

- Vorzeichen (Sign): 1 Bit – positiv (0) oder negativ (1)
- Summe: 52 + 11 + 1 = 64 Bit (IEEE 754 Double Precision)

ZEICHENKODIERUNG

Zeichen werden als numerische Codes im Speicher abgelegt.

Computer arbeiten intern ausschließlich mit Zahlen – daher muss jedes Zeichen (Buchstabe, Ziffer, Symbol) einer eindeutigen Zahl zugeordnet werden.

Beispiele für Zeichenkodierungsstandards:
- ASCII – ordnet 128 Zeichen Zahlen von 0 bis 127 zu
- Unicode – umfasst über 140.000 Zeichen aus allen Schriftsystemen weltweit
- UTF-8 – eine weit verbreitete Kodierung, die Unicode-Zeichen mit variabler Byte-Länge darstellt

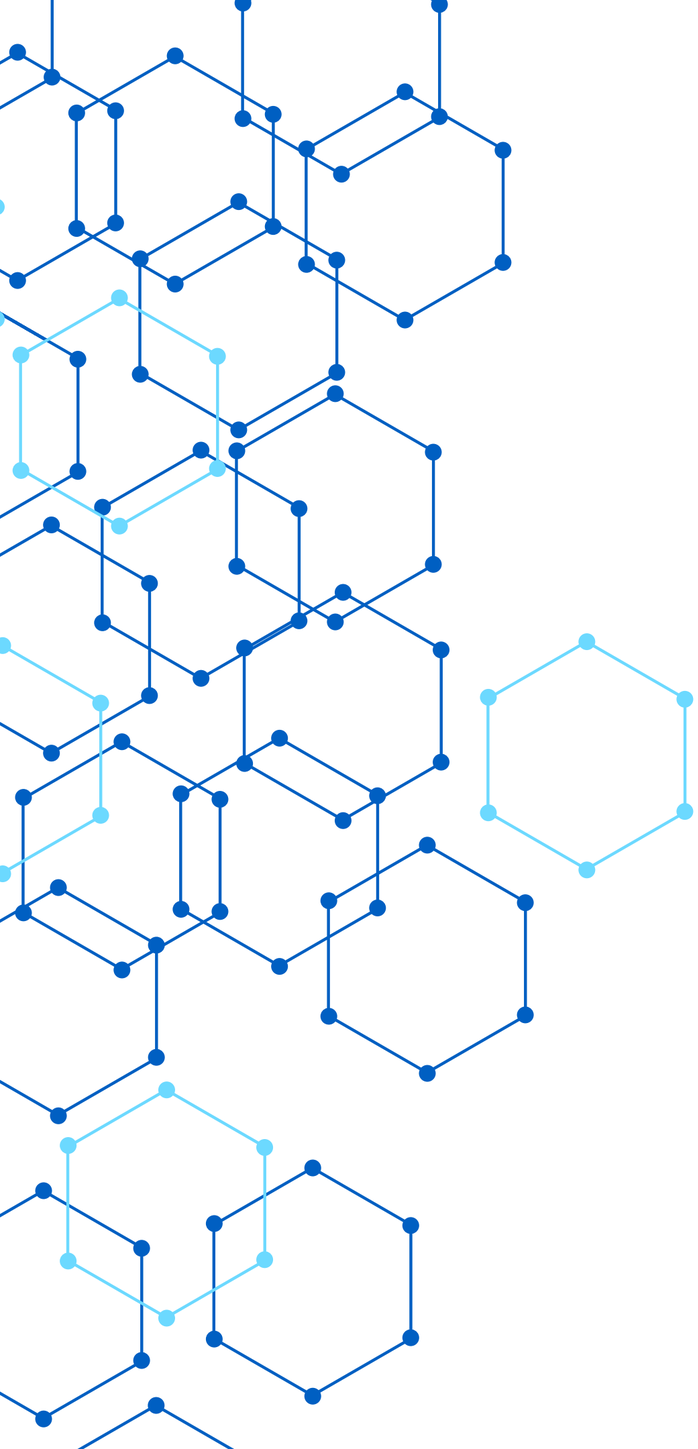
In Python wird standardmäßig UTF-8 verwendet.

VISUAL STUDIO CODE INSTALLIEREN

Visual Studio Code (VS Code) ist ein kostenloser, leistungsstarker Code-Editor, der für die Python-Entwicklung empfohlen wird.

Installation unter: https://code.visualstudio.com

Bei Fragen oder Problemen stehe ich sehr gerne zur Verfügung



ERSTES PROGRAMM SCHREIBEN

Nachdem die Entwicklungsumgebung eingerichtet ist, kann das erste Python-Programm geschrieben werden.

Möglichkeiten zum Starten:
– Direkt im Terminal über den interaktiven Python-Interpreter
– In einem Editor wie Visual Studio Code als .py-Datei

Erstes klassisches Programm:
print("Hello, World!")

Die Datei wird im Terminal mit folgendem Befehl ausgeführt:
python dateiname.py

Dieser erste Schritt vermittelt das grundlegende Prinzip:
Code schreiben → speichern → ausführen → Ergebnis sehen.

TERMINALBEFEHLE

Verzeichnis wechseln
cd ~ → Wechselt ins Home-Verzeichnis.
cd src → Wechselt in den Ordner "src".
cd .. → Wechselt ein Verzeichnis nach oben (übergeordnetes Verzeichnis).

Aktuelles Verzeichnis anzeigen & Inhalte listen
pwd → Zeigt den vollständigen Pfad des aktuellen Verzeichnisses.
ls → Listet Dateien und Ordner.
ls sub → Listet Inhalte des Unterordners "sub".
ls -la → Detaillierte Liste inkl. versteckter Dateien und Berechtigungen.

Hilfe aufrufen
man ls → Öffnet das Handbuch (Manual) zum Befehl ls. Mit "man" lässt sich zu jedem Terminal-Befehl eine ausführliche Dokumentation aufrufen. Beenden mit der Taste q.

PYTHON INTERAKTIV AM TERMINAL

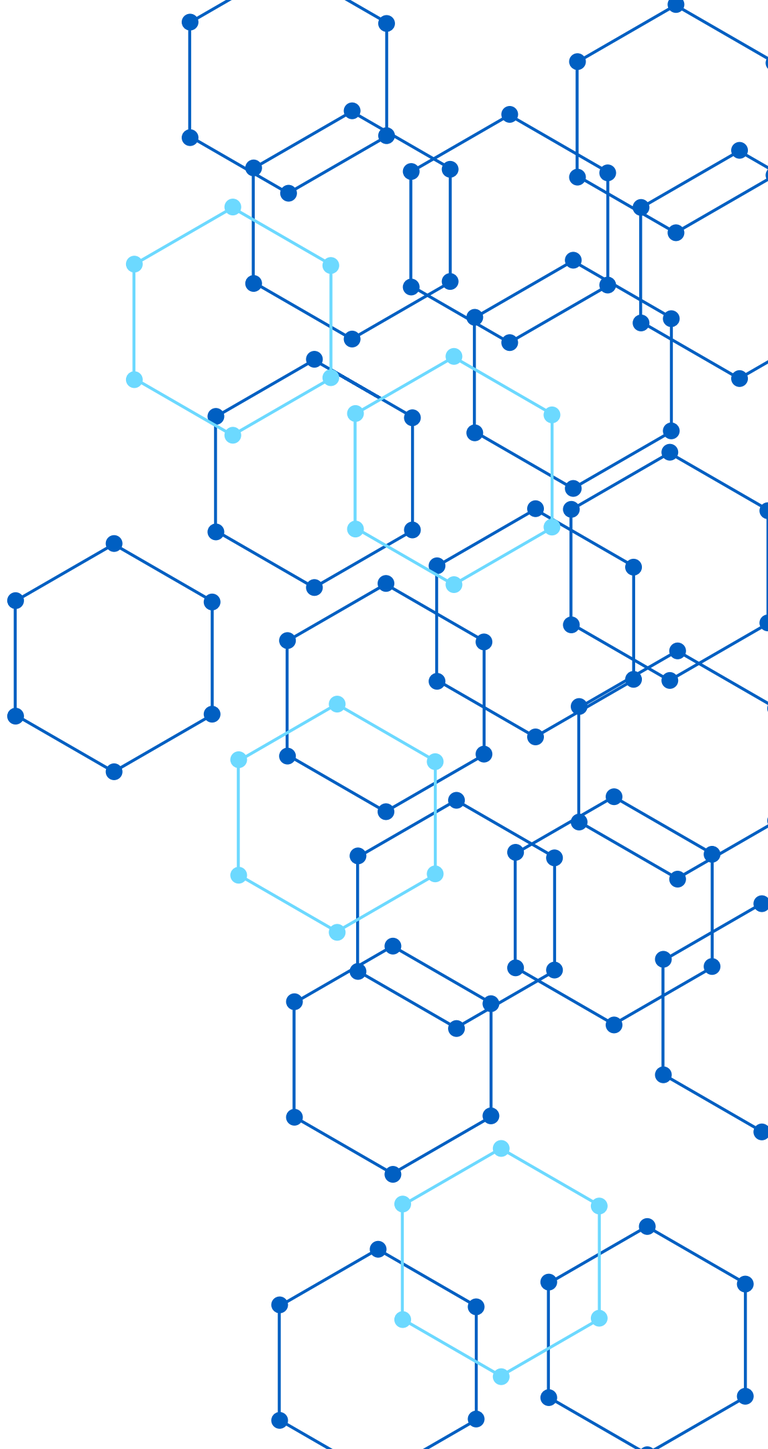
Python im interaktiven Modus starten:
→ Befehl: python3
→ Der Python-Interpreter startet und zeigt den Prompt: >>>

Erste Eingaben im REPL (Read–Eval–Print Loop):

```
2 + 3          # Ausgabe: 5
print("Hallo")  # Ausgabe: Hallo
type(42)       # Ausgabe: <class 'int'>
```

Interaktiven Modus beenden:
→ exit()

Hinweis: Der interaktive Modus eignet sich ideal zum schnellen Testen von Ausdrücken, Funktionen und Datentypen – ohne eine Datei speichern zu müssen.



EINFACHE DATENSTRUKTUREN & OPERATOREN

Grundlegende Datentypen
bool: True / False (1 / 0)
int: Ganze Zahlen beliebiger Länge
float: Nachkommazahl, 64-Bit
str: Zeichenketten (Strings)

Arithmetische & Zuweisungsoperatoren
// | % | ** | += | -=
Klammerung beeinflusst Reihenfolge
Mischung int/float ergibt float

Vergleichs- & logische Operatoren
!= | <= | >=
and, or, not
Liefern stets Wahrheitswerte (bool)
in-Operator für Strings & Sequenzen

ARITHMETISCHE OPERATIONEN & TYPUMWANDLUNG

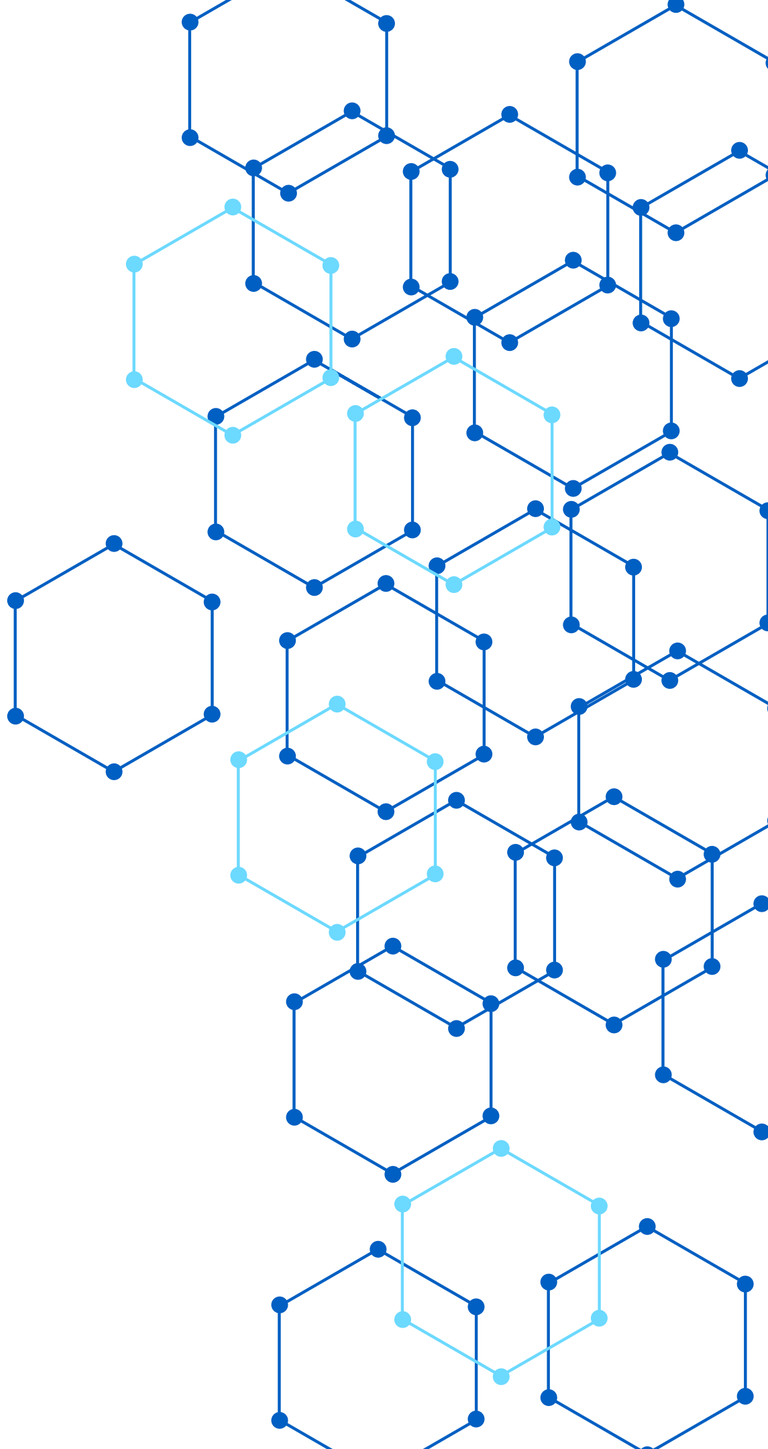
Operatoren & Beispiele:
2**3 → 8 (Potenzierung)
(2+3)*4 → 20 (Klammerung)
7/2 → 3.5 (echte Division)
7//2 → 3 (Ganzzahldivision)
7%2 → 1 (Modulo)
divmod(7,2) → (3,1)

Typumwandlung:
int + float → float (z.B. 1 + 2.0 → 3.0)

Vergleichsoperatoren:
3 > 2 → True | 3 == 2 → False

Strings:
"Hallo" + " Welt" → "Hallo Welt"
"all" in "Hallo" → False

Fehler:
1/0 → ZeroDivisionError: division by zero



VARIABLEN UND DYNAMISCHE TYPISIERUNG

- Variablen speichern Objekte – nicht direkt Werte, sondern Referenzen auf Objekte im Speicher. Operationen mit Variablen sind möglich, solange die Typen kompatibel sind.
- Typfehler bei inkompatiblen Typen: z. B. führt die Addition von int und str zu einem TypeError. Python prüft die Typen zur Laufzeit, nicht zur Kompilierzeit.
- Eingebaute Funktionen: type(x) gibt den Datentyp eines Objekts zurück. id(x) gibt die eindeutige Speicheradresse (Identität) des Objekts zurück.

TYPISIERUNG VON VARIABLEN

Variablen als Referenzen
In Python sind Variablen keine Behälter, die Werte direkt speichern – sie sind Referenzen (Zeiger) auf Objekte im Speicher. Der Bezeichner verweist auf ein Objekt, das einen bestimmten Typ besitzt. Mit id(x) lässt sich die Speicheradresse des referenzierten Objekts ermitteln.

Typ kann sich ändern – Dynamische Typisierung
Python ist dynamisch typisiert: Eine Variable ist nicht an einen festen Typ gebunden. Der Typ ergibt sich aus dem aktuell referenzierten Objekt. Beispiel: n = 3 → n ist vom Typ int. Danach: n = 'Hallo' → n ist nun vom Typ str. Der Typ wechselt zur Laufzeit automatisch mit der Zuweisung.

VIELEN DANK!

Fragen & Diskussion – wir freuen uns auf euren Austausch!